

同步方法之二



了解同步方法的技术特性

北京理工大学计算机学院
金旭亮

完成一些准备工作

//封装一些辅助代码以便重用

6 usages

```
public class ThreadHelper {  
    3 usages  
    private final String name;  
    2 usages  
    public ThreadHelper(String name) {  
        this.name = name;  
    }  
    4 usages  
    public ThreadHelper(){  
        this.name="线程" + Thread.currentThread().getName();  
    }  
}
```

6 usages

```
public void process() {  
    for (int i = 0; i < 4; i++) {  
        System.out.println(name + "正在处理" + i);  
        try {  
            //用于拖慢执行速度, 以方便演示  
            Thread.sleep( millis: 500);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

这个类主要就是提供一个可以由多线程执行的代码，以避免重复编写，并且使用name字段值拼接出有意义的提示信息。

定义测试用类

//此类定义了一些同步方法，主要用于展示同步方法的技术特性

```
public class SynchronizedClass {  
    //非同步方法  
    1 usage  
    public void nonSyncMethod(){  
        new ThreadHelper().process();  
    }  
    //同步方法  
    6 usages  
    public synchronized void syncMethod(){  
        new ThreadHelper().process();  
    }  
}
```

//两个指定了名字的同步方法

```
1 usage  
public synchronized void syncMethod1(){  
    new ThreadHelper( name: "syncMethod1").process();  
}  
1 usage  
public synchronized void syncMethod2(){  
    new ThreadHelper( name: "syncMethod2").process();  
}
```

//两个同步静态方法

```
2 usages  
public synchronized static void syncStaticMethod(){  
    new ThreadHelper().process();  
}  
1 usage  
public synchronized static void syncOtherStaticMethod(){  
    new ThreadHelper().process();  
}
```

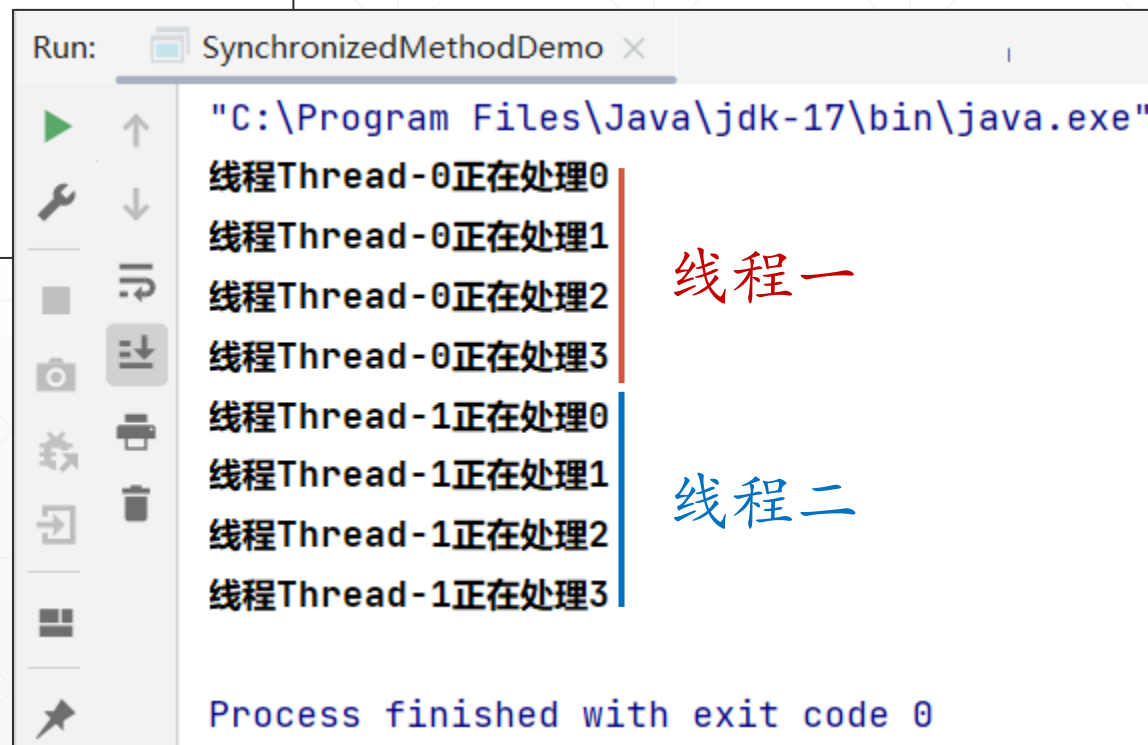
进行测试-1

//多线程访问同一个实例的同一个同步方法

1 usage

```
private static void multiThreadInvokeOneSyncMethod() {  
    var obj = new SynchronizedClass();  
    new Thread(obj::syncMethod).start();  
    new Thread(obj::syncMethod).start();  
}
```

从输出结果中可以看到，两个线程是“排队”调用共享对象的同步方法的。



```
Run: SynchronizedMethodDemo x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
线程Thread-0正在处理0  
线程Thread-0正在处理1  
线程Thread-0正在处理2  
线程Thread-0正在处理3  
线程Thread-1正在处理0  
线程Thread-1正在处理1  
线程Thread-1正在处理2  
线程Thread-1正在处理3  
Process finished with exit code 0
```

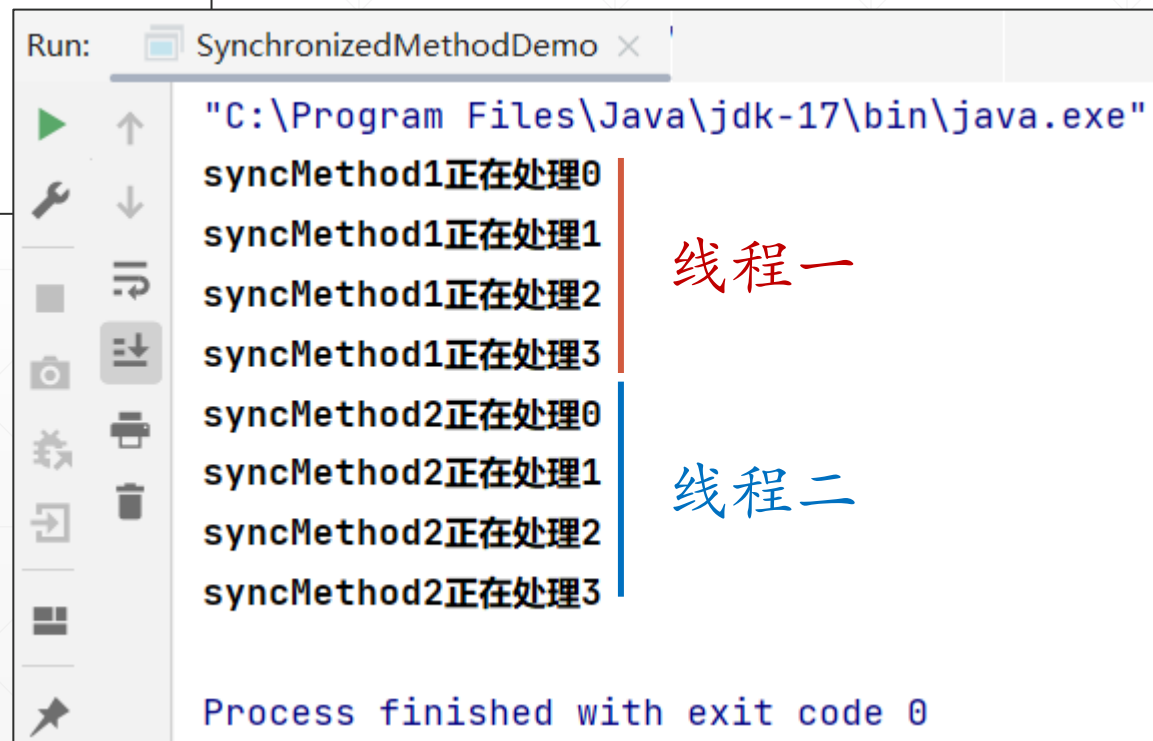
进行测试-2

//多线程访问同一个实例的不同同步方法

1 usage

```
private static void multiThreadInvokeMultiSyncMethod() {  
    var obj = new SynchronizedClass();  
    new Thread(obj::syncMethod1).start();  
    new Thread(obj::syncMethod2).start();  
}
```

从输出可以看到，多线程执行同一个对象的不同同步方法，也是“排好队”顺序调用的，不会交替执行。



```
Run: SynchronizedMethodDemo x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
syncMethod1正在处理0  
syncMethod1正在处理1  
syncMethod1正在处理2  
syncMethod1正在处理3  
syncMethod2正在处理0  
syncMethod2正在处理1  
syncMethod2正在处理2  
syncMethod2正在处理3  
  
Process finished with exit code 0
```

进行测试-3

//多线程访问同一个实例的同步与不同步方法

1 usage

```
private static void multiThreadInvokeSyncAndNonSyncMethod() {  
    var obj = new SynchronizedClass();  
    new Thread(obj::nonSyncMethod).start();  
    new Thread(obj::syncMethod).start();  
}
```

两个线程，一个调用同步方法，另一个调用普通方法，代码是交替执行的。



Run: SynchronizedMethodDemo x

"C:\Program Files\Java\jdk-17\bin\java.exe"

线程Thread-0正在处理0
线程Thread-1正在处理0
线程Thread-0正在处理1
线程Thread-1正在处理1
线程Thread-1正在处理2
线程Thread-0正在处理2
线程Thread-1正在处理3
线程Thread-0正在处理3

Process finished with exit code 0

线程一与线程
二交替执行

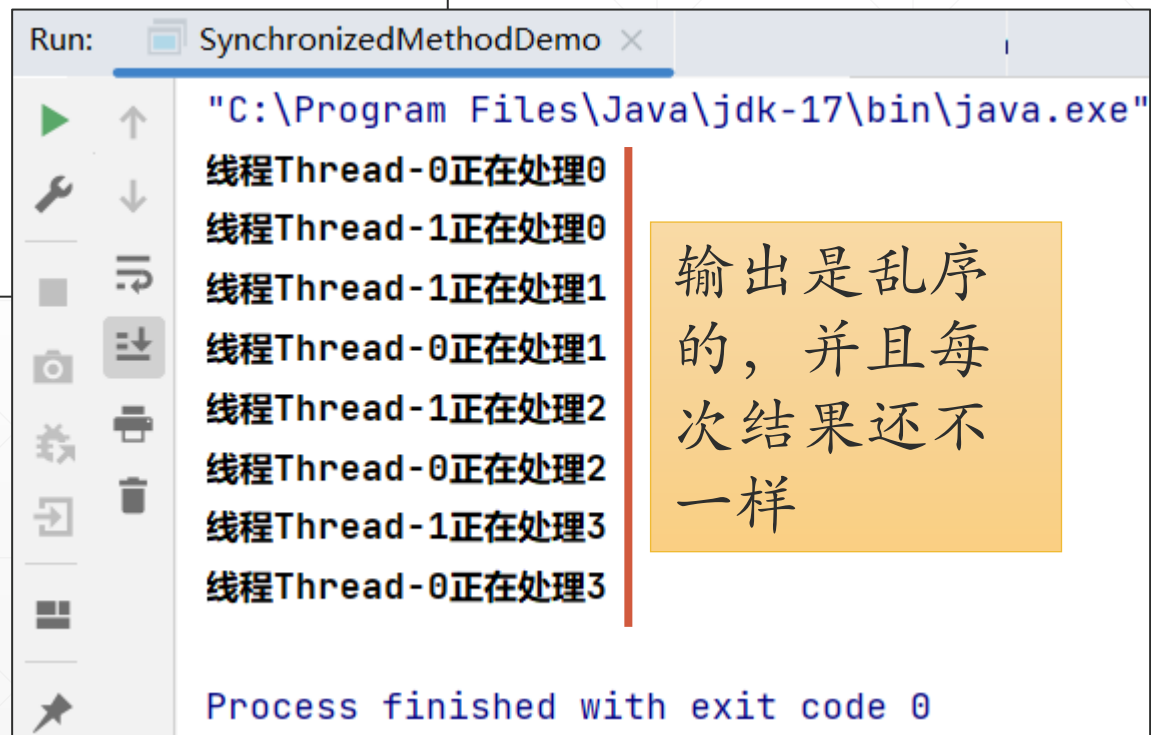
进行测试-4

//多线程访问不同实例的同一个同步方法

1 usage

```
private static void multiThreadInvokeMultiObjSameSyncMethod() {  
    var obj1 = new SynchronizedClass();  
    new Thread(obj1::syncMethod).start();  
    var obj2 = new SynchronizedClass();  
    new Thread(obj2::syncMethod).start();  
}
```

多线程访问不同实例的同步方法，代码是交替执行的，没有“同步”的效果。



```
Run: SynchronizedMethodDemo x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
线程Thread-0正在处理0  
线程Thread-1正在处理0  
线程Thread-1正在处理1  
线程Thread-0正在处理1  
线程Thread-1正在处理2  
线程Thread-0正在处理2  
线程Thread-1正在处理3  
线程Thread-0正在处理3  
  
Process finished with exit code 0
```

输出是乱序的，并且每次结果还不一样

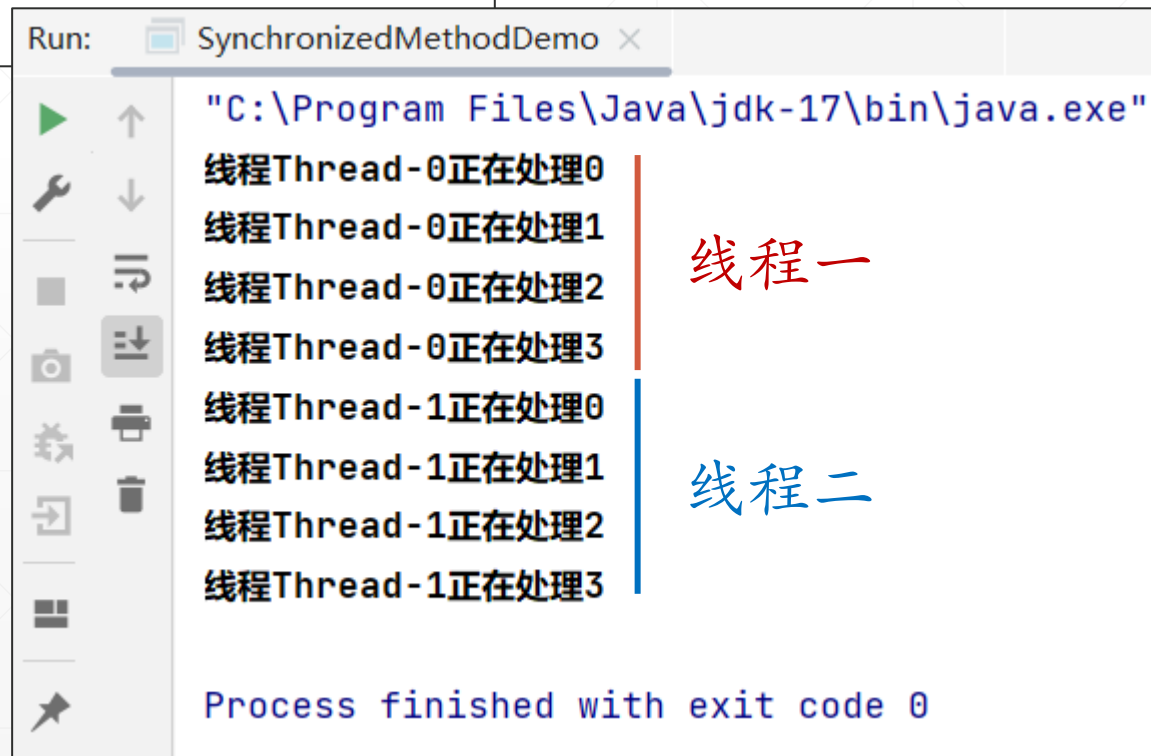
进行测试-5

//多个线程顺序访问不同的同步静态方法

1 usage

```
private static void multiThreadInvokeMutliSyncStaticMethod() {  
    new Thread(SynchronizedClass::syncStaticMethod).start();  
    new Thread(SynchronizedClass::syncOtherStaticMethod).start();  
}
```

对于静态同步方法，多线程访问它们，总是“排好队”顺序访问的。



```
Run: SynchronizedMethodDemo x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
线程Thread-0正在处理0  
线程Thread-0正在处理1  
线程Thread-0正在处理2  
线程Thread-0正在处理3  
线程Thread-1正在处理0  
线程Thread-1正在处理1  
线程Thread-1正在处理2  
线程Thread-1正在处理3  
Process finished with exit code 0
```


进行测试-6



//多线程访问同一个类的同步静态与同步实例方法

1 usage

```
private static void multiThreadInvokeSyncStaticAndInstanceMethod() {  
    var obj = new SynchronizedClass();  
    new Thread(obj::syncMethod).start();  
    new Thread(SynchronizedClass::syncStaticMethod).start();  
}
```

多线程访问同一个类的同步静态或实例方法，代码是交替执行的，没有“同步”效果。

Run: SynchronizedMethodDemo x

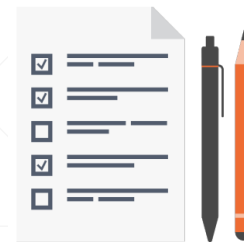
"C:\Program Files\Java\jdk-17\bin\java.exe"

线程Thread-1正在处理0
线程Thread-0正在处理0
线程Thread-1正在处理1
线程Thread-0正在处理1
线程Thread-1正在处理2
线程Thread-0正在处理2
线程Thread-1正在处理3
线程Thread-0正在处理3

Process finished with exit code 0

线程一与线程二乱序执行

结论



多个线程访问同一个对象的同步方法，或者是同一个类的同步静态方法，总是“排好队”顺序访问。



多个线程访问同一个类所定义的同步静态方法和实例方法，或者访问不同实例的同步实例方法，没有同步效果。



造成上述现象的根本原因，在于“锁定的对象”不一样，只有锁定相同对象的线程，才能同步。